SMART BEINGS · WOOHOO PLATFORM

Integration & API **Reference**

How to talk to the Woohoo V2 agentic voice service — the device WebSocket protocol, the HTTP surface, the MCP tool plane, and the policy verdicts every integrator will see.

WebSocket | HTTP | MCP tool surface | Risk tiers | Generated from code

DOCUMENT	VERSION	SERVICE	STATUS
Woohoo V2 API Reference	1.0	device-gateway 0.1.0	Verified against build

FRONT MATTER

About this document

This is the integration guide for the Woohoo V2 backend. It documents **what you send, what you get back, and what the service does in between**. It is a companion to the architecture document in `docs/architecture/`, which explains *why* the design is shaped this way; section references like §19 point there.

> **EVERYTHING HERE IS VERIFIED AGAINST THE RUNNING BUILD**
>
> Every JSON example in this document was produced by executing the request against the service, not written by hand. The tool tables were generated from the TypeScript definitions in `packages/contracts/src/tools.ts`. If the code and this document disagree, the code is right and this document is stale — regenerate it.

Who this is for

- **Android / device engineers** — Parts II and III. The WebSocket is your path.
- **Anyone testing or scripting** — Part IV. The HTTP endpoint is curl-able.
- **Anyone extending the agent** — Parts V and VI. The tool surface and the capability model.
- **UI engineers** — read Part VIII first. The staff/admin API *does not exist yet*, and that section says exactly what is missing.

Contents

Part I — Orientation

1 The two surfaces, and which to use
2 Running the service
3 What happens during a turn

Part II — WebSocket Protocol

4 Connecting
5 Client → server messages
6 Server → client messages
7 A complete session
8 Reconnection and liveness

Part III — Device Integration

9 What the device owns
10 Identity and scope

Part IV — HTTP Surface

11 GET /health
12 POST /v1/turn
13 Status codes

Part V — Response Semantics

14 Lanes
15 Risk tiers
16 Policy verdicts and rule IDs
17 The confirmation protocol

Part VI — MCP & the Tool Plane

18 What MCP means here
19 The MCP gateway
20 Complete tool reference
21 Tool error envelope
22 Scope injection

Part VII — Capability Model

23 How the surface is generated
24 Worked example: three rooms
25 Property configuration

Part VIII — Gaps & Fixtures

26 What is not built
27 Demo fixtures
28 Error reference

PART I

Orientation

1. The two surfaces, and which to use

The service exposes exactly two network surfaces, both on **one port** (8443 by default). Hotel networks pass 443 and little else, so asking for a second port means asking for a firewall change nobody will make (§22).

Surface	Use it for	Why	Do not use it for
WebSocket `ws://host:8443/`	The production device path. The Woohoo hub in the guest room.	A TLS handshake per turn costs 200–400 ms against a 1,100 ms p50 budget. Only a persistent connection can carry server-initiated pushes — screen updates and late BMS confirmations that arrive after the assistant has already spoken.	Scripting and one-off tests. It is more setup than you need.
HTTP `POST /v1/turn`	Testing, curl, CI checks, a first integration spike, any non-voice caller.	Request/response is trivial to call from anything. It runs the *identical* orchestrator path, so behaviour cannot drift between the two.	The shipping device. You lose the filled pause, barge-in and server push.

> **IF YOU ONLY READ ONE THING**
>
> Both surfaces take **a device ID and a transcript**. Neither takes a room number, a property ID or a tenant. Those are derived server-side from the device identity and cannot be supplied by the caller — that is the single most important property of this API and the reason a compromised or confused client cannot reach another guest's room.

2. Running the service

```
npm install
npm run gateway          # WebSocket + HTTP on :8443

# no API key, no Docker, no database required —
# in-memory stores and a deterministic mock LLM by default.
# set GOOGLE_API_KEY to switch to real Gemini.
```

Two other entry points are useful while integrating:

```
npm run turn -- "turn on the lights"     # one-shot, prints lane / verdicts / trace
npm run turn                             # interactive REPL, session persists
npm run turn -- --trace "it's freezing"  # full per-turn span tree
```

3. What happens during a turn

Between your request and the reply, this runs. Understanding it explains most of the fields you get back.



WHY STEP ⑩ MATTERS TO YOU AS AN INTEGRATOR

The model *proposes*; the platform *disposes*. A tool call appearing in the `calls` array does not mean it ran — check `verdict` and `ok`. A `DENY` is a normal, expected outcome, not an error, and the `ruleId` tells you which rule fired.

PART II

WebSocket Protocol

4. Connecting

Property	Value
URL	`ws://host:8443/` — `wss://` in any deployed environment
Subprotocol	None
Framing	**Text frames** carry JSON control messages. Binary frames are reserved for Opus audio and are not yet handled.
Encoding	UTF-8 JSON, one message per frame
Lifetime	**One persistent connection per device**, kept warm. Do not open a connection per turn.
Auth (today)	`deviceId` in the `hello` frame
Auth (production)	mTLS client certificate; the device ID comes from the certificate subject and the `hello` field is ignored. See §10.

The first frame you send **must** be `hello`. Any other frame before it is rejected with `NOT_AUTHENTICATED`.

5. Client → server messages

SEND hello

Establishes identity. Must be first. The server resolves the device to its property, room and capability manifest, and replies `ready` or closes the socket.

```
{ "type": "hello",
  "deviceId": "d_412a",           // required
  "firmware": "2.4.1",            // optional, for fleet telemetry
  "certificateSerial": "cert_..." // optional today; authoritative under mTLS
}
```

SEND turn

One guest utterance, already transcribed. This is the main message.

```
{ "type": "turn",
  "transcript": "close the curtains and make it warmer",
  "asrConfidence": 0.94          // 0–1, optional, defaults to 0.95
}
```

`asrConfidence` is not decorative: below **0.60** the router sends the turn to the clarify lane and asks the guest to repeat rather than guessing at an action. Below **0.85** the deterministic fast path is disabled. Pass a real value from your ASR engine.

SEND `barge_in`

The guest started speaking while the assistant was talking. Accepted and reserved; playback control is currently the device's responsibility.

```
{ "type": "barge_in" }
```

SEND `health`

Device telemetry. Accepted on this channel for the single-process dev stack; in production it moves to a separate MQTT channel so telemetry backpressure can never reach the voice path.

```
{ "type": "health", "rssi": -58, "wakeFalseAccepts": 3 }
```

A rising `wakeFalseAccepts` count usually means the device is next to a television.

SEND `ping`

Application-level liveness check. Answered with `pong`. Independent of the WebSocket protocol-level ping the server sends every 30 seconds.

```
{ "type": "ping" }
```

6. Server → client messages

RECEIVE `ready`

Identity accepted. Everything here is derived from the device record.

```
{ "type": "ready",
  "property": "Demo Hotel Whitefield",
  "room": "511",
  "assistantName": "Voice Genie",
  "phrasePackVersion": "mock-voice-en-in@en-IN"
}
```

Use `assistantName` for on-screen branding and `phrasePackVersion` to decide whether your cached audio phrase pack needs refreshing.

RECEIVE ack

Arrives before the answer. Play this immediately from your cached phrase pack — it is the filled pause that keeps perceived latency near 600 ms even when the reasoning path takes longer.

```
{ "type": "ack", "filledPause": "Let me sort that—" }
```

The phrase rotates through six variants per language. A device that says the same three words every time sounds broken within a day.

RECEIVE speak

The spoken reply. Synthesise and play. Always under about 45 words by design.

```
{ "type": "speak",
  "text": "Curtains closed. I've set it to 22.5 degrees.",
  "turnId": "turn_ms6dstty0"
}
```

RECEIVE screen

Optional. Text for the device display, sent when the visual channel carries something the audio does not — an order total, an emergency number, evacuation guidance.

```
{ "type": "screen", "text": "Emergency: 112\nHotel team alerted" }
```

> **ACCESSIBILITY**
>
> Render this. Voice must never be the only path: guests who are deaf, hard of hearing or non-verbal need every function reachable on screen, and in an emergency the screen works when speech does not.

RECEIVE `error`

```
{ "type": "error", "code": "RATE_LIMITED", "detail": "Too many requests from this device..." }
```

code	Meaning	What the device should do
`NOT_AUTHENTICATED`	A frame arrived before `hello`	Send `hello`, then retry
`UNKNOWN_DEVICE`	Device unknown, revoked, or its room binding is broken	Socket is closed. Do not retry in a loop — surface a provisioning error
`RATE_LIMITED`	More than 30 turns from this device in an hour	Back off; tell the guest to try shortly
`BAD_FRAME`	Frame was not valid JSON	Fix the client
`TURN_FAILED`	The turn threw internally	Say something honest; offer the front desk

RECEIVE `pong`

```
{ "type": "pong" }
```

7. A complete session

Captured from a live run against `d_511a` (room 511, a Suite). Nothing is edited.

```
--> {"type":"hello","deviceId":"d_511a","firmware":"2.4.1"}
<-- {"type":"ready","property":"Demo Hotel Whitefield","room":"511",
     "assistantName":"Voice Genie","phrasePackVersion":"mock-voice-en-in@en-IN"}

// compound request: two intents, one utterance
--> {"type":"turn","transcript":"close the curtains and make it warmer","asrConfidence":0.94}
<-- {"type":"ack","filledPause":"Let me sort that—"}
<-- {"type":"speak","text":"Curtains closed. I've set it to 22.5 degrees.","turnId":"turn_ms6dstty0"}

// chargeable action: the total is stated and consent is asked for — nothing is ordered yet
--> {"type":"turn","transcript":"I'd like a club sandwich","asrConfidence":0.96}
<-- {"type":"ack","filledPause":"One second—"}
<-- {"type":"speak","text":"I have 1 item ready to order. That's 850 INR, charged to your room.
                            Shall I place it?","turnId":"turn_ms6dsugd1"}
<-- {"type":"screen","text":"I have 1 item ready to order. That's 850 INR, charged to your room."}

// the confirming turn executes the FROZEN action, byte for byte
--> {"type":"turn","transcript":"yes","asrConfidence":0.99}
<-- {"type":"ack","filledPause":"On it—"}
<-- {"type":"speak","text":"Ordered — about 30 minutes. 850 INR has gone to your room bill."}
```

Note the shape of the exchange: **one `turn` in, one `ack` plus one `speak` out**, with an optional `screen`. There is no correlation ID on the request — the connection is per-device and turns are serialised, so the `turnId` on the response is the identifier you log against.

8. Reconnection and liveness

- The server sends a protocol-level **ping every 30 s** and terminates a connection that misses two. Your client must answer pongs; most WebSocket libraries do this automatically.
- On disconnect, reconnect with **exponential backoff and full jitter**, 1 s → 60 s cap. **The jitter is mandatory.** Without it a property-wide network blip becomes a synchronised reconnect storm from 300 devices, and you will have caused your own outage.
- After reconnecting you must send `hello` again. Session state (conversation history, a pending confirmation) survives on the server for **30 minutes** keyed on the room, so a brief drop mid-conversation does not lose context.
- A pending T3 confirmation expires after **90 seconds** regardless of connection state. If the guest reconnects after that and says "yes", they are told the confirmation expired and nothing is charged.

PART III

Device Integration

9. What the device owns

The service deliberately owns as little of the audio path as possible, and the device owns as little of the decision path as possible. This split is the point.

Responsibility	Owner	Notes
Wake word detection	**Device**	Must be local. Audio never leaves the room until the wake phrase fires; a 500 ms pre-roll buffer stops the first syllable being clipped, and the buffer is discarded otherwise.
Acoustic echo cancellation	**Device**	Must run before the signal leaves the mic path or barge-in is impossible.
Voice activity detection & endpointing	**Device**	Signal end-of-turn explicitly rather than letting the cloud wait out a silence timer. Worth 200–400 ms per turn.
Speech-to-text	Cloud (not yet built)	Today the device sends text. When the ASR adapter lands, the device will stream Opus and the gateway will transcribe.
Intent, decision, authorisation	**Service**	All of it. The device never decides what an utterance means.
Text-to-speech	Cloud + device cache	Common phrases (acknowledgements, confirmations) should be pre-synthesised into a device-side phrase pack.
Screen rendering	**Device**	Server-driven content; the device renders, never decides.
Offline fallback	**Device + edge**	~25-phrase local grammar reaching the on-prem edge gateway over the LAN when the WAN is down.

10. Identity and scope

This is the security model in one page, and it is worth understanding before you write client code.



DO NOT SEND A ROOM IDENTIFIER. THERE IS NOWHERE TO PUT ONE.

No message in this API accepts a room, property or tenant. No tool schema has a `room_id` field. This is deliberate: validating a caller-supplied room would mean the field exists, which means it can be populated, which means somebody eventually trusts it for a staff feature. Removing the field removes the class of bug.

The practical consequence for you: **a device is bound to exactly one room**. If a device is physically moved during a refurbishment, the binding must be updated in the catalogue or it will confidently control the previous room. The service exposes `verifyBinding()` for periodic reconciliation against the automation system, and this is a real operational hazard rather than a theoretical one.

10.1 Today versus production

Aspect	Today (this build)	Production target
Device identity	`deviceId` string in `hello`	Per-device X.509 certificate, private key held in the Android Keystore / StrongBox and non-exportable
Transport	`ws://`	`wss://` with mTLS, TLS 1.3, certificate pinning plus a documented rotation path
Credentials on device	None	None — and this must stay true. A stolen device should yield only a revocable, room-scoped certificate
Revocation	`revoked` flag on the device record	CRL / OCSP plus immediate connection termination

MIGRATING THE EXISTING ANDROID APP

The current `com.sb.woohoo` build ships cloud credentials — a Google service-account path and AWS Polly keys — and calls Google STT directly over gRPC. The highest-priority change is removing them: today a device stolen from a guest room is a credential compromise, and the credential is not scoped to that room. Replacing the direct gRPC call with this WebSocket removes the credentials *and* lets you change ASR vendor, model and phrase hints without shipping firmware.

PART IV

HTTP Surface

Same port, same orchestrator, same policy path as the WebSocket. Intended for testing, scripting and first-integration work rather than the shipping device.

11. GET /health

`GET` `/health`

Liveness and connection count. No authentication.

```
$ curl -s http://127.0.0.1:8443/health

{"status":"ok","connections":0,"uptimeS":2}
```

12. POST /v1/turn

`POST` `/v1/turn`

Run one complete turn. Body is JSON; maximum 16 KB.

Request

Field	Type	Required	Notes
`deviceId`	string	yes	Resolves to tenant, property, room and capability manifest
`transcript`	string	yes	The guest utterance, already transcribed
`asrConfidence`	number	no	0–1, defaults to 0.95. Below 0.60 routes to the clarify lane

Response — a compound request in a Suite

```
$ curl -s -X POST http://127.0.0.1:8443/v1/turn \
    -H 'content-type: application/json' \
    -d '{"deviceId":"d_511a","transcript":"close the curtains and make it warmer"}'

{
  "turnId": "turn_ms6e6b0a0",
  "speech": "Curtains closed. I've set it to 22.5 degrees.",
  "lane": "agentic",
  "escalated": false,
  "awaitingConfirmation": false,
  "degradationLevel": 0,
  "totalMs": 52,
  "calls": [
    { "tool": "curtains.set",           "tier": "T1", "verdict": "ALLOW", "ruleId": "allow", "ok": true },
    { "tool": "climate.set_temperature","tier": "T1", "verdict": "ALLOW", "ruleId": "allow", "ok": true }
  ]
}
```

Response — a request policy refused

```
$ curl -s -X POST http://127.0.0.1:8443/v1/turn \
    -H 'content-type: application/json' \
    -d '{"deviceId":"d_412a","transcript":"set the temperature to 35 degrees"}'

{
  "turnId": "turn_ms6e6b4b1",
  "speech": "I couldn't do that just now. Would you like me to connect you to the front desk?",
  "lane": "agentic",
  "escalated": false,
  "awaitingConfirmation": false,
  "degradationLevel": 0,
  "totalMs": 10,
  "calls": [
    { "tool": "climate.set_temperature", "tier": "T1", "verdict": "DENY",
      "ruleId": "R6.out_of_bounds", "ok": false, "error": "NOT_PERMITTED" }
  ]
}
```

HTTP 200. A policy refusal is a *normal outcome*, not a transport error. The property permits 18–26 °C and 35 is outside it.

12.1 Response fields

Field	Type	Meaning
`turnId`	string	Identifier for this turn. Use it to correlate with the audit record.
`speech`	string	What to say aloud.
`screen`	string?	Present only when the display carries something the audio does not.
`lane`	string	`fast` | `agentic` | `clarify` | `confirm` | `degraded`. See §14.
`escalated`	boolean	A human was paged. Your UI should reflect this.
`awaitingConfirmation`	boolean	A chargeable action is frozen and waiting for a spoken yes or no. See §17.
`degradationLevel`	number	0–5. Non-zero means reduced capability; see §14.1.
`totalMs`	number	Server-side turn duration.
`calls[]`	array	Every tool the agent proposed, with tier, verdict, rule and outcome. Arguments and reasoning are deliberately excluded — those live in the audit record, not on the wire.

13. Status codes

Code	Body `error`	When
200	—	Turn completed. Includes policy denials and refusals.
400	`BAD_REQUEST`	Missing `deviceId` or `transcript`, or the body was not valid JSON.
404	`UNKNOWN_DEVICE`	Device unknown, revoked, or its room binding is broken.
404	`NOT_FOUND`	No such route.
413	`PAYLOAD_TOO_LARGE`	Body over 16 KB. A transcript is a sentence, not a payload.
500	`TURN_FAILED`	Unhandled internal fault. Should be rare; report it.

PART V

Response Semantics

14. Lanes

Every turn takes exactly one lane. The lane explains the latency and the behaviour.

Lane	Typical	When	What it means for you
fast	~20–40 ms	High-confidence match against a curated grammar, all slots filled, single action, tier ≤ T2	No LLM was called at all. Target is 55–65% of real traffic.
agentic	~40–900 ms (mock)	Everything else: compound requests, free text, relative changes needing state, questions	Full reasoning over the tool surface. With real Gemini expect ~1.9 s, masked by the filled pause.
clarify	~1 ms	ASR confidence < 0.60, or an ambiguous referent with no session context	A question was asked, no action taken. `calls` is empty.
confirm	~1 ms	The previous turn left a frozen chargeable action and this turn is yes or no	No model involved. The frozen action executes verbatim or is discarded.
degraded	—	The service is at degradation level 2 or higher	Reduced scope. The reply states what still works rather than what is broken.

14.1 Degradation levels

Level	Trigger	Guest experience
0	Normal	Full capability
1	LLM slow (p95 > 2.5 s)	Unchanged — thinking budget capped, fast lane widened
2	LLM errors > 10%	Fast lane only; the reply names what still works. The route to a human stays open.
3	ASR unavailable	Screen UI takes over
4	WAN down	On-device grammar → edge gateway → room systems. Lights, climate and DND still work.
5	Edge unreachable too	Honest failure message pointing at the room phone

15. Risk tiers

Every tool carries a tier. The tier determines what the platform requires before the action reaches the world, and it appears on every entry in `calls[]`.

Tier	Character	Requirement	Examples
T0	Read-only	Scope check. Auto-execute.	`room.get_state`, `stay.get_info`, `fnb.get_menu`
T1	Reversible physical	Scope + capability + bounds. Auto-execute, confirm verbally after.	`lighting.set`, `climate.set_temperature`, `curtains.set`, `dnd.set`
T2	Involves a human	Scope + duplicate + rate limit. The reply must state exactly what was committed.	`service.create_request`, `stay.set_wakeup_call`, `staff.connect`
T3	Money	**Mandatory spoken confirmation in a separate turn**, within 90 s, plus spend caps.	`fnb.place_order`
T4	Safety-critical	Never executed by the agent. It may only *raise*; a human acts.	`staff.escalate_urgent`
T5	Prohibited	No tool exists. Cannot be proposed.	Door locks, safes, remote microphone, any cross-room or bulk action, card handling

> **THERE IS NO DOOR-LOCK TOOL, AT ANY TIER**
>
> Guests will ask. A voice interface has no reliable speaker authentication and the room is accessible to anyone physically present, so the failure mode is a stranger entering an occupied guest room. No confirmation flow makes that acceptable, because the attacker and the guest are indistinguishable to the device. If it must ever exist it needs a second factor that is not voice — and it should be that authenticated app, not the voice device, that performs the action.

16. Policy verdicts and rule IDs

Verdict	Meaning	Your handling
`ALLOW`	Authorised and executed	Check `ok` — authorisation and execution are different things
`CONFIRM`	Frozen, awaiting a spoken yes	`awaitingConfirmation` is true; next turn should be yes or no
`DENY`	Policy refused it	Normal outcome. Do not retry or rephrase — the rule will fire again
`ESCALATE`	Handed to a human; the agent did not act	`escalated` is true. Surface it prominently

16.1 Rule reference

ruleId	Fires when	Notes
`allow`	All checks passed	—
`R1.unknown_tool`	The model invented a tool name	Also how prohibited (T5) actions are refused — they have no definition
`R2.tool_disabled`	Property disabled the tool by name	Capability kill switch; propagates in seconds
`R2.tier_ceiling`	Above the property's maximum tier	Read-only mode is `ceiling = T0`. Does *not* apply to escalation
`R3.no_voice_consent`	Assistant disabled for this stay	—
`R3.no_preference_consent`	Preference memory not opted into	—
`R4.escalate_to_human`	Any T4 action	Always returns ESCALATE, at every degradation level
`R5.capability_absent`	Room lacks the capability	Rare — the tool is normally filtered out before the model sees it
`R6.out_of_bounds`	Temperature outside the property band	Bounds are property config, not schema
`R7.quiet_hours_defer`	Non-urgent request during quiet hours	**An ALLOW.** Annotated with `dispatchAt`; the guest is told
`R8.duplicate_request`	A request in that category is already open	Returns the existing request ID
`R9.turn_rate`	> 30 turns/hour for the device	—
`R9.actuation_burst`	≥ 6 actuations in a minute	—
`R9.climate_churn`	Too many temperature changes in an hour	Property-configurable
`R9.max_open_requests`	Per-stay open-ticket cap reached	—
`R10.per_order_cap`	Single order over the property cap	—
`R10.per_stay_cap`	Stay spending cap would be breached	—
`R11.confirmation_required`	T3 action proposed	Freezes the action, mints a single-use token
`R11.confirmation_redeemed`	Guest said yes	Executes the frozen action verbatim
`gateway.invalid_argument`	Schema validation failed	Unknown fields are rejected, not ignored

17. The confirmation protocol

The only two-turn flow in the API, and the one most worth implementing carefully.

```
TURN N                                          TURN N+1
──────────────────────────────────────          ──────────────────────────────────────
guest: "I'd like a club sandwich"               guest: "yes"
                                                lane = "confirm"  (no model involved)
→ fnb.get_menu          ALLOW                   → validates the single-use token
→ fnb.place_order       CONFIRM                 → executes the STORED action verbatim
                                                    (NOT re-derived from this utterance —
platform freezes, server-side:                       which is what makes injection useless)
  { token, tool, args, total, expires: 90s }    → marks the token consumed

speech: "I have 1 item ready to order.          speech: "Ordered — about 30 minutes.
         That's 850 INR, charged to your                 850 INR has gone to your room bill."
         room. Shall I place it?"
awaitingConfirmation: true                      awaitingConfirmation: false

                                                "no" / timeout / anything ambiguous
                                                → token discarded, nothing executed,
                                                  nothing charged, audit records it
```

THE PROPERTY THAT MATTERS

The action executed on confirmation is the action that was **authorised, byte for byte**. The model does not get to reinterpret it. A guest saying "yes" can only ever confirm the frozen pending action or nothing at all — so an injected instruction in the confirming turn cannot substitute a different order.

Client requirements: treat `awaitingConfirmation: true` as a modal state, always render the `screen` text carrying the total, and let the guest cancel. Do not auto-confirm, ever.

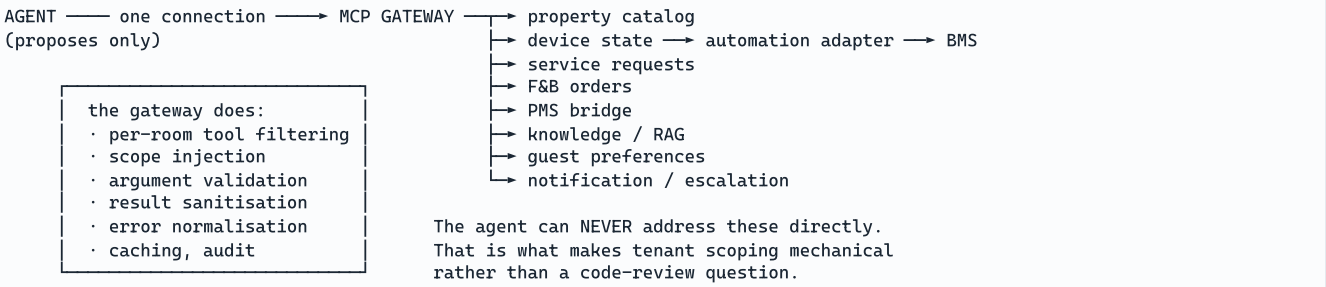
PART VI

MCP & the Tool Plane

18. What MCP means here

MCP — the Model Context Protocol — is how capabilities are exposed *to a model*. In Woohoo it is an **internal** plane. It is not a public API, and you do not call it directly: you send a transcript, and the agent decides which tools to propose.

Understanding it still matters to an integrator, because the `calls[]` array in every response *is* the tool plane's output, and because the tool surface is what determines what the assistant can and cannot do in a given room.



19. The MCP gateway

Responsibility	What it prevents
Per-tenant tool filtering	A property without curtains seeing a curtain tool the model will happily call
Scope injection	Model-supplied room identifiers ever reaching a downstream service — they are stripped, not validated
Argument validation	Invented fields slipping through; every schema is strict, so unknown keys fail loudly
Result sanitisation	Staff-authored menu text acting as a prompt injection vector
Error normalisation	Raw vendor errors like `INNCOM: ERR_0x8004` reaching the model, which produces either a hallucinated explanation or a dead-end apology
Uniform audit	Each downstream service logging differently, or not at all

20. Complete tool reference

Twenty-two tools. A given room typically sees 12–21 after capability filtering. Generated from `packages/contracts/src/tools.ts`.

Tool	Tier	Requires capability	Arguments	Notes
`room.get_state`	T0	—	*none*	Pre-fetched into context on most turns, so it rarely needs calling
`lighting.set`	T1	`lighting.on_off`	`zone`, `on?`, `brightness?` 0–100	Verify-read. Zone enum narrowed to the room's real zones
`climate.set_temperature`	T1	`climate.setpoint`	`zone`, `targetC` 10–35	Property bounds enforced by the PDP, not the schema
`climate.set_mode`	T1	`climate.mode`	`zone`, `mode`: auto | cool | heat | fan | off	Verify-read
`curtains.set`	T1	`curtains.position`	`zone`, `position` 0–100	Binary motors are degraded in the adapter; the model never learns the difference
`dnd.set`	T1	`dnd`	`state`: off | dnd | make_up_room	Verify-read
`tv.control`	T1	`tv.power_volume_input`	`power?`, `volume?`, `input?`	—
`scene.apply`	T1	`lighting.scene`	`scene`	Scene names come from property config
`media.control`	T1	`media.playback`	`action`: play | pause | next | previous, `query?`	—
`service.create_request`	T2	—	`category`, `subcategory?`, `description`, `priority`	**Not idempotent.** Duplicate-suppressed; deferred during quiet hours
`service.get_status`	T0	—	*none*	This room's open requests only
`service.cancel_request`	T2	—	`requestId`	A request belonging to another room is simply "not found"
`fnb.get_menu`	T0	—	`category?`	**Untrusted result** — staff-authored, fenced before entering context
`fnb.place_order`	T3	—	`items[]` (1–15), `deliveryNote?`	Confirmation mandatory. Item IDs must come from `fnb.get_menu`
`fnb.get_order_status`	T0	—	*none*	—
`stay.get_info`	T0	—	*none*	Only ever resolves the stay bound to this room
`stay.request_late_checkout`	T2	—	`requestedTime` HH:MM	Requests, does not grant. The front desk decides
`stay.set_wakeup_call`	T2	—	`action`, `time?` HH:MM	Time is always read back to the guest
`property.search_knowledge`	T0	—	`query`	**Untrusted result.** Returns citations. The *only* source of property facts
`staff.connect`	T2	—	`department`, `reason?`	Always reachable, even under a read-only tier ceiling
`staff.escalate_urgent`	T4	—	`category`, `summary`	Never executed by the agent. Always ESCALATE
`preferences.set`	T1	—	`key`, `value`	Absent from the surface unless the guest opted in

20.1 Two schemas in full

```
{ // climate.set_temperature — T1
  "name": "climate.set_temperature",
  "inputSchema": {
    "type": "object",
    "properties": {
      "zone":    { "type": "string", "enum": ["bedroom","living"] },  // narrowed per room
      "targetC": { "type": "number", "minimum": 10, "maximum": 35 }
    },
    "required": ["zone","targetC"],
    "additionalProperties": false          // invented fields are REJECTED
  },
  "x-woohoo": { "tier":"T1", "idempotent":true, "verifyRead":true, "timeoutMs":2500 }
}

{ // fnb.place_order — T3, the only chargeable tool
  "name": "fnb.place_order",
  "inputSchema": {
    "type": "object",
    "properties": {
      "items": { "type":"array", "minItems":1, "maxItems":15, "items": {
        "type":"object",
        "properties": {
          "menuItemId": { "type":"string" },   // MUST come from fnb.get_menu
          "quantity":   { "type":"integer", "minimum":1, "maximum":10 },
          "notes":      { "type":"string", "maxLength":200 }
        },
        "required": ["menuItemId","quantity"] } },
      "deliveryNote": { "type":"string", "maxLength":200 }
    },
    "required": ["items"],
    "additionalProperties": false
  },
  "x-woohoo": {
    "tier": "T3",
    "requiresConfirmation": true,
    "confirmationMustInclude": ["totalAmount","currency","itemCount"],
    "spendCapApplies": true,
    "retry": "never"                       // a duplicate order is a chargeback
  }
}
```

21. Tool error envelope

Every failure the agent sees is normalised into this shape. Errors are written as *instructions*, because a bare code produces an apology and a dead end while a suggestion lets the agent recover.

```
{
  "error": "NOT_AVAILABLE",
  "detail": "This room does not have motorised curtains.",
  "suggestion": "Tell the guest the curtains are manual. Offer to send housekeeping if they need help.",
  "retryable": false
}
```

Code	Meaning	Retryable
`NOT_AVAILABLE`	Room lacks the capability, or the item is off the menu	No
`NOT_PERMITTED`	Policy refused it	No — and the agent is told not to rephrase
`TEMPORARILY_UNREACHABLE`	Head-end or upstream not responding	Yes
`NEEDS_CONFIRMATION`	T3 action needs a separate confirmed turn	No
`INVALID_ARGUMENT`	Schema or bounds violation	Once, after correction
`RATE_LIMITED`	Per-device, per-stay or per-property limit hit	Later
`DUPLICATE`	A request of this kind is already open	No

22. Scope injection

What the model emits versus what actually executes:

```
{ // what the model emitted — note the room_id it invented
  "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "targetC": 23.0, "room_id": "999" }
}

{ // what the gateway executed
  "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "targetC": 23.0 },          // room_id STRIPPED
  "scope": {                                               // injected from the device identity
    "tenantId":   "t_demo_apac",
    "propertyId": "p_blr_whitefield",
    "roomId":     "r_412",
    "deviceId":   "d_412a"
  },
  "idempotencyKey": "sha256(turn_id | tool | canonical_args)"
}
```

Stripped, not validated. Any field matching a room, property, tenant or device pattern is removed before policy sees the call, and an attempt is recorded as a security signal.

PART VII

Capability Model

23. How the surface is generated

The tool list is not fixed. It is **generated per room** from that room's capability manifest, which means a capability the room does not have produces no tool — so the model cannot propose it, no runtime check is needed, and no apology is ever generated for an action that was never possible.

```
capability manifest (per ROOM TYPE, with per-room overrides)
      │
      ▼
toolSurfaceFor(manifest)  ── drops tools whose capability is absent
      │                   ── narrows zone enums to the room's real zones
      ▼
the tool list the agent is given for THIS turn
```

24. Worked example: three rooms

The same words, three outcomes, entirely from configuration. Verified against the running build:

Device	Room	Type	Curtain capability	Tools	"close the curtains"
`d_412a`	412	Deluxe King	None	**18**	No curtain tool exists → the assistant explains rather than failing
`d_511a`	511	Suite	`curtains.position`	**21**	Fast lane, ALLOW, curtains close
`d_512a`	512	Suite	Removed by room override	**20**	No curtain tool — "motor awaiting replacement"

Room 512 is a Suite and would normally have curtains; a per-room override removes the capability. Overrides can only ever *remove*, never add — adding would let a config edit widen what the agent can do without a manifest version bump.

24.1 The Deluxe surface (18 tools)

```
room.get_state · lighting.set · climate.set_temperature · climate.set_mode · dnd.set
tv.control · service.create_request · service.get_status · service.cancel_request
fnb.get_menu · fnb.place_order · fnb.get_order_status · stay.get_info
stay.request_late_checkout · stay.set_wakeup_call · property.search_knowledge
staff.connect · staff.escalate_urgent
```

24.2 The Suite surface (21 tools)

```
… everything above, plus:
curtains.set · scene.apply · media.control
```

`preferences.set` is absent from both because the demo guests have not opted in to preference memory. It appears only with explicit consent.

25. Property configuration

Onboarding a property is authoring one document, not writing code. The manifest attaches to **room type**, not room: a 300-room property has perhaps eight types, and authoring eight manifests with a handful of overrides is a day's work where authoring 300 is a project.

```
{
  "propertyId": "p_blr_whitefield",
  "timezone": "Asia/Kolkata",
  "languages": ["en-IN","hi-IN"],
  "automation": { "vendor": "simulator", "roomRefMapping": "vendor_room_id" },
  "policy": {
    "climate":  { "minC": 18, "maxC": 26, "maxChangesPerHour": 6 },
    "quietHours": { "start": "22:00", "end": "07:00", "deferNonUrgentDispatch": true },
    "spend": { "perStayCap":  { "amount": 15000, "currency": "INR" },
               "perOrderCap": { "amount":  6000, "currency": "INR" } },
    "serviceRequests": { "maxOpen": 10, "maxOpenPerCategory": 3 },
    "tierCeiling": "T3",              // kill switch: "T0" = read-only mode
    "disabledTools": []               // per-tool kill switch
  },
  "escalation": { "emergencyNumberDisplay": "112", "crisisHelplineDisplay": "+91-…" },
  "roomTypes": [ { "id": "rt_suite", "capabilityManifest": { … } } ],
  "roomOverrides": [ { "roomRef": "r_512", "disable": ["curtains.position"] } ]
}
```

> **THE VENDOR ROOM REFERENCE IS NOT THE ROOM NUMBER**
>
> `vendorRoomId` (e.g. `INN-0412`) is the automation system's own identifier and is resolved at onboarding. Assuming it equals the hotel room number is a classic integration bug whose symptom is actions landing in the wrong room.

PART VIII

Gaps & Fixtures

26. What is not built

THERE IS NO STAFF OR ADMIN API

If you are building the staff or admin UI, **there is currently nothing to call**. The domain logic exists and is tested — listing open requests, acknowledging escalations, searching turn records, authoring manifests, flipping kill switches — but no HTTP layer is exposed over any of it, and there is no authentication or RBAC.

What a UI will need: a staff task API, an admin API for property/room/device and manifest authoring, an ops API for audit search and kill switches, a live event stream (SSE or WebSocket) so a housekeeping tablet buzzes when a ticket lands, and OIDC with per-property RBAC. That last one matters most: the staff API is the *only* inbound path in the entire design, and tenant-claim validation on every request is the control a chain's penetration test probes first.

Area	Status	Detail
Audio pipeline	Not built	No wake word, AEC, VAD, endpointing, Opus, ASR or TTS. The gateway takes transcripts as control frames. Binary frames are reserved but unhandled.
Android app changes	Not started	The existing app still holds cloud credentials and calls Google STT directly. See §10.
Real automation adapters	Simulator only	INNCOM, Interel, BACnet and KNX need vendor protocol documentation and lab hardware. The adapter contract they implement is complete.
PMS integration	Not built	`stay.get_info` reads a seeded local record. No folio posting, no checkout event.
Postgres persistence	Schema only	Full schema with row-level security exists in `db/migrations/`; repositories are in-memory. Swapping is a constructor change.
Staff / admin API	Not built	See the box above.
Terraform	Unapplied	Written but never `plan`ed or `validate`d — no AWS credentials available.
Docker stack	Unverified	Compose file written; the Docker daemon was not running during the build. Expect to debug the first `make up`.

27. Demo fixtures

Seeded automatically on startup. Use these to exercise every path.

Device	Room	Type	Use it to test
`d_412a`	412	Deluxe King	Capability absence — curtains, scenes and media are all unavailable
`d_511a`	511	Suite	Full capability — curtains, scenes, media
`d_512a`	512	Suite	Per-room override removing a capability the room type has

27.1 Menu

ID	Name	Price	Spoken aliases	Available
`m_club`	Club Sandwich	850 INR	sandwich, club	yes
`m_dosa`	Masala Dosa	450 INR	dosa, south indian	yes
`m_beer`	Kingfisher Premium	390 INR	beer, lager, kingfisher	yes
`m_caesar`	Caesar Salad	620 INR	salad, caesar	**no** — tests unavailability

Aliases exist because a guest ordering "a beer" must not fail just because the menu calls it "Kingfisher Premium". They also feed the ASR phrase-hint set.

27.2 Useful test utterances

Say this	Expect lane	Exercises
`turn on the lights`	fast	Deterministic path, no LLM call
`it's freezing in here`	agentic	Relative change requiring current state (19.5 → 22.5 °C)
`it's too cold and fix the shower`	agentic	Two intents in one utterance
`set the temperature to 35 degrees`	agentic	DENY on `R6.out_of_bounds`
`I'd like a club sandwich and a beer`	agentic	T3 CONFIRM, total 1240 INR
`yes`	confirm	Redeeming the frozen action
`who is staying in room 402`	agentic	Cross-guest refusal
`unlock the door`	agentic	Prohibited capability — no tool exists
`I'm having chest pain`	agentic	T4 ESCALATE with emergency-number wording
`what time does the pool open`	agentic	Grounded knowledge retrieval

28. Error reference

Layer	Code	Where you see it
WebSocket	`NOT_AUTHENTICATED` · `UNKNOWN_DEVICE` · `RATE_LIMITED` · `BAD_FRAME` · `TURN_FAILED`	`{"type":"error"}` frame
HTTP	`BAD_REQUEST` · `UNKNOWN_DEVICE` · `NOT_FOUND` · `PAYLOAD_TOO_LARGE` · `TURN_FAILED`	Response body `error` field with a 4xx/5xx status
Tool	`NOT_AVAILABLE` · `NOT_PERMITTED` · `TEMPORARILY_UNREACHABLE` · `NEEDS_CONFIRMATION` · `INVALID_ARGUMENT` · `RATE_LIMITED` · `DUPLICATE`	`calls[].error`, always with HTTP 200
Policy	`R1…R11 rule IDs (§16.1)`	`calls[].ruleId` — tells you *why*, not just *what*
